



ДОКУМЕНТАЦИЯ · ВЕРСИЯ 5.27.2

Спецификация приложения

Назначение, роли и права, функции разделов, модель данных, правила расчётов, серверный API, безопасность и развёртывание в сети без интернета.

ВЕРСИЯ
5.27.2

ПЛАТФОРМА
Vite 8 · React 19 · SQL Server

СРЕДА
корпоративная сеть, IIS

ДАТА
04.10.2026

00 Содержание

01	Назначение и область применения	зачем нужно приложение и где работает
02	Термины	словарь предметной области
03	Пользователи, роли и разрешения	роли, 20 разрешений, видимость задач по иерархии
04	Функциональные требования	экраны и поведение каждого раздела
05	Модель данных	сущности, поля и ограничения
06	Бизнес-правила и расчёты	недели, сроки, баллы, остатки, пересечения
07	Серверная часть	API, выполнение действий, синхронизация
08	Хранилища данных	SQL Server, PostgreSQL, файл
09	Аутентификация и безопасность	токены, вход через Windows, защита данных
10	Уведомления	события, адресаты, напоминания
11	Интерфейс и оформление	библиотека компонентов, темы, адаптивность
12	Нефункциональные требования	сеть, браузеры, целостность, версии
13	Развёртывание	IIS без интернета, переменные, Vercel
14	Разработка, проверки и версионирование	стек, команды, проверки, выпуски
15	Известные ограничения	что приложение не делает или делает иначе
16	Приложения	права вкладок, данные браузера, шаблоны, демо-данные

Как читать. Главы 01–03 — что это за приложение и кто с ним работает. Главы 04–06 — что оно делает и по каким правилам. Главы 07–10 — как устроена серверная часть. Главы 11–15 — оформление, требования и публикация. В приложениях — справочные таблицы. Источник истины — код; описание собрано по версии 5.27.2.

Документ описывает приложение целиком: назначение, пользователей, функции каждого раздела, модель данных, бизнес-правила и расчёты, серверный API, безопасность, развёртывание и требования к среде. Источник истины — код; где в тексте указан файл, правило реализовано в нём. Пользовательское руководство — `docs/manual.html` (PDF и раздел «Информация»), порядок публикации в корпоративной сети — `docs/corporate-offline.md`.

01 Назначение и область применения

«Контроль задач» — веб-приложение для отдела: планирование задач, контроль сроков исполнения, недельная отчётность, график событий сотрудников (отпуска, командировки и т. п.) и расчёт показателей эффективности по баллам.

Основная идея. Задача заводится один раз и сразу видна во всех разделах: в календаре, в недельном плане, на доске сроков, в отчёте и в карточке сотрудника. Руководитель планирует и оценивает, исполнитель ведёт исполнение своих задач.

Поток данных.

- 1 Руководитель создаёт задачу (Календарь, Планирование или Контроль).
- 2 Исполнитель описывает результат и отмечает исполнение.
- 3 Руководитель при необходимости меняет баллы задачи.
- 4 Кнопка «Направить в отчёт» фиксирует исполненные задачи недели и их баллы в отчёте.
- 5 Баллы из отчётов попадают в «Показатели» и карточки сотрудников.

Среда эксплуатации.

- Рабочая среда — **корпоративная сеть без доступа в интернет**: IIS + iisnode, Microsoft SQL Server, вход через Windows (Kerberos/NTLM). Приложение не обращается к внешним службам, шрифтам и CDN.
- Демонстрационная среда — Vercel + Neon PostgreSQL (только для показа; не является рабочей).
- Целевой браузер — Google Chrome (в том числе 109, последний для Windows 7) и Edge 109.

Вне области. Мобильное приложение, интеграция с внешними почтовыми и мессенджер-службами, офлайн-режим клиента без сервера (работа без сервера есть только в тестах и в режиме `VITE_BACKEND=local`).

02 Термины

ТЕРМИН	ЗНАЧЕНИЕ
Задача (мероприятие)	Единая запись: название, раздел планирования, категория, исполнители, начало, окончание (срок), документ, результат, отметка об исполнении, баллы.
Раздел планирования (позиция плана)	Элемент иерархического справочника с кодом вида 3.1.2. Раздел (isHeader) — заголовок, содержит вложенные; позиция — рабочая единица с базовым весом в баллах. Задача привязывается к позиции.
Категория	Значение справочника «Категории задач»; определяет цвет задачи в календаре. Задача без категории показывается как «Иное».
Плановая неделя	Пятница — четверг. Используется в «Планировании» и «Отчётности». В неделю попадают задачи, срок которых лежит в [пятница; следующая пятница].
Календарная неделя	Понедельник — воскресенье. Используется в «Календаре» и «Контроле».
Баллы	Вес исполненной задачи. Индивидуальные баллы задачи, иначе базовый вес позиции. Диапазон 0,1–15, шаг 0,1.
Отчёт	Снимок исполненных задач плановой недели с баллами на момент отправки. Один отчёт на неделю; повторная отправка заменяет предыдущий.
Отсутствие (событие)	Отпуск, командировка, отгул, больничный, учёба. В разделе «Тетрис» называется «событие», в карточке сотрудника — «отсутствие».
Заявка	Отсутствие со статусом request, ожидает решения руководителя.
Подразделение	Иерархия «организация → управление → отдел → отделение». Отделения основного отдела — группы в фильтрах.
Пересечение	Неисполненная задача, срок которой приходится на отсутствие её исполнителя.
Комплект IIS	Папка dist-iis (сервер server.cjs, статика public\, web.config, setup-sql.ps1, data\) — результат npm run build:iis.

03 Пользователи, роли и разрешения

3.1. Роли

Три роли заданы в системе (`administrator` , `manager` , `executor`). Состав разрешений каждой роли настраивается администратором и хранится в базе. Все права проверяются **на сервере** тем же кодом (`src/lib/reducer.ts`), что и в браузере; интерфейс лишь скрывает недоступное.

роль	назначение	разрешения по умолчанию
Администратор	Настраивает систему и доступ; может всё, что руководитель	Все разрешения
Руководитель	Планирует, оценивает, согласует отсутствия	<code>reports.view</code> , <code>tasks.plan</code> , <code>tasks.execute</code> , <code>tasks.score</code> , <code>tasks.delete</code> , <code>absences.manage</code> , <code>absences.request</code> , <code>entitlements.manage</code> , <code>appearance.manage</code>
Исполнитель	Ведёт исполнение своих задач, подаёт заявки на отсутствие	<code>tasks.execute</code> , <code>absences.request</code> , <code>appearance.manage</code>

3.2. Справочник разрешений

разрешение	содержание	группа
<code>admin.access</code>	Открывать администрирование	Администрирование
<code>users.manage</code>	Пользователи: вести и назначать роли	Администрирование
<code>units.manage</code>	Подразделения: вести дерево	Администрирование
<code>roles.manage</code>	Роли и разрешения: настраивать	Администрирование
<code>authentication.manage</code>	Аутентификация: настраивать способ входа	Администрирование
<code>scoring.manage</code>	Оценка: настраивать правила подсчёта баллов	Администрирование
<code>dictionaries.manage</code>	Словари: редактировать значения	Администрирование
<code>planRows.manage</code>	Разделы планирования: вести план	Администрирование
<code>templates.manage</code>	Шаблоны документов: редактировать	Администрирование
<code>appearance.manage</code>	Редактирование UI: настраивать своё оформление	Администрирование
<code>reports.view</code>	Просматривать отчётность и показатели	Работа отдела
<code>tasks.plan</code>	Планировать задачи для сотрудников	Задачи
<code>tasks.execute</code>	Вести исполнение назначенных задач	Задачи

РАЗРЕШЕНИЕ	СОДЕРЖАНИЕ	ГРУППА
<code>tasks.score</code>	Назначать баллы	Задачи
<code>tasks.viewAll</code>	Видеть задачи всего отдела (иначе — свои и своего подразделения)	Задачи
<code>tasks.delete</code>	Удалять задачи	Задачи
<code>absences.manage</code>	Вести отсутствия и согласовывать заявки	Отсутствия
<code>absences.request</code>	Подавать заявки на отсутствие	Отсутствия
<code>entitlements.manage</code>	Изменять нормы отпусков и отгулов	Отсутствия
<code>data.reset</code>	Восстанавливать демонстрационные данные	Система

Версия набора прав. Константа `ACCESS_VERSION = '7'`. Если в базе отметка отстаёт, при чтении администратору дописываются права `authentication.manage`, `scoring.manage`, `units.manage`, `planRows.manage`, `templates.manage`, `tasks.viewAll`, а всем ролям — `appearance.manage`. Отметка хранится в `Data.accessVersion` (файл) либо в таблице `tc_meta` (SQL Server, PostgreSQL).

3.3. Матрица возможностей по умолчанию

ДЕЙСТВИЕ	РУКОВОДИТЕЛЬ	ИСПОЛНИТЕЛЬ
Видеть задачи	свои и своего подразделения (начальник отделения); начальник отдела и администратор — все	только свои
Создавать задачи	любому сотруднику	только себе (исполнитель принудительно подставляется, индивидуальные баллы сбрасываются)
Менять название, сроки, позицию, категорию, исполнителей	да	нет
Документ, результат, отметка об исполнении	в любой задаче	только в назначенных ему
Баллы за исполнение в карточке задачи	видит и меняет	не видит; свои баллы — в «Моей карточке»
Удалять задачи	да	нет
Планирование	все сотрудники, фильтры по группе и сотруднику, отправка отчёта	только свой столбец
Контроль	все, фильтр по сотруднику	только свои задачи

ДЕЙСТВИЕ	РУКОВОДИТЕЛЬ	ИСПОЛНИТЕЛЬ
Отчётность, Показатели	да	раздел скрыт, прямая ссылка ведёт в календарь
Карточки сотрудников	все	только своя («Моя карточка»)
Тетрис	график отдела, остатки всех	график отдела, остатки только свои
Отсутствия	создаёт и меняет любому, сразу «согласовано»	заявка только на себя; менять и отзывать до решения
Согласовать или отклонить заявку	да	нет
Нормы отпуска и отгулов	меняет	видит свои
Восстановить демоданные	только администратор	нет
Администрирование	по разрешениям вкладок; администратор — все	вкладка «Редактирование UI»

3.4. Правила доступа к задаче и отсутствию

`taskAccess ( src/lib/permissions.ts )`:

- Есть `tasks.plan` → правка плана задачи; исполнение — при `tasks.execute`; баллы — при `tasks.score`; удаление — при `tasks.delete`.
- Нет `tasks.plan`:
 - новая задача — разрешено планирование и исполнение, баллы и удаление запрещены;
 - существующая — исполнение только если пользователь среди исполнителей.

`absenceAccess` :

- `absences.manage` → править, решать и удалять любые.
- Только `absences.request` → создавать заявки; менять и удалять **только собственную** заявку в статусе `request`.

3.5. Видимость задач по иерархии

Правило в `src/lib/visibility.ts`; его применяют сервер и редьюсер, поэтому оно не зависит от интерфейса.

КТО	ОБЛАСТЬ ВИДИМОСТИ
Есть <code>tasks.viewAll</code> (администратор, начальник отдела)	все задачи отдела
Есть <code>tasks.plan</code> , но нет <code>tasks.viewAll</code> (начальник отделения)	сотрудники его подразделения (<code>unitId</code>) вместе с вложенными; задача видна, если среди исполнителей есть кто-то из них
Остальные (подчинённые)	только свои задачи

- **Сервер** (`forUser` в `server/app.ts`) отдаёт клиенту только видимые задачи, а в отчётах — только строки видимых сотрудников; чужие задачи не попадают в браузер вообще.
- **Редьюсер** не даёт изменить или удалить невидимую задачу и назначить исполнителем сотрудника вне области (уже назначенных вне области оставить можно, иначе нельзя было бы править видимую задачу). Исполнитель по-прежнему создаёт задачи только себе.
- **Уведомление** «Задача исполнена» получают только те руководители, которым задача видна.
- **Интерфейс**: фильтры «Исполнитель»/«Сотрудник»/«Группа», выбор исполнителей в карточке задачи, столбцы «Планирования», дерево «Сотрудников» (карточка вне области недоступна) и «Показатели» ограничены областью видимости.
- Пользователь без подразделения видит только свои задачи. Заместитель начальника отдела видит весь отдел, если роли выдано `tasks.viewAll`.

3.6. Защита от потери административного доступа

Нельзя отключить или удалить последнего активного пользователя, чья роль содержит одновременно `admin.access`, `users.manage` и `roles.manage`; нельзя удалить собственную учётную запись; сохранение прав роли, после которого такого пользователя не остаётся, отклоняется. Сервер при загрузке дополнительно восстанавливает доступ (`recoverAdministrativeAccess`): возвращает роль администратора по умолчанию и резервного администратора `user@example.com`, если иначе войти некому.

04 Функциональные требования

4.1. Общая структура интерфейса

- Адреса: `/login`, `/calendar` (по умолчанию), `/planning`, `/control`, `/tetris` (прежний `/absences` перенаправляется), `/reports`, `/kpi`, `/employees`, `/employees/:id`, `/chat`, `/info`, `/kit`, `/admin`; остальное — страница «Страница не найдена».
- Без входа любой адрес ведёт на `/login`; после входа открывается запрошенная страница, иначе календарь.
- `/reports`, `/kpi` требуют `reports.view`; `/admin` — доступности хотя бы одной вкладки администрирования; иначе переход в календарь.
- Шапка закреплена: название, имя и роль вошедшего, служебные кнопки, ниже — разделы. Пункты меню: Календарь, Планирование, Контроль, Тетрис, Отчётность*, Показатели*, Сотрудники (у исполнителя — «Моя карточка»), Переписка, Информация. *Только при `reports.view`.
- Служебные кнопки: колокольчик (число просроченных задач — у исполнителя только его; ведёт в «Контроль»), шестерёнка (администрирование), солнце/луна (тема), круговая стрелка (восстановление демоданных, только с `data.reset`, подтверждается вторым нажатием), квадрат с инициалами (своя карточка), выход.
- Логотип — галочка из квадратных блоков; ведёт в календарь; при наведении блоки по очереди «подпрыгивают» (при `prefers-reduced-motion` — только мягкое мигание).
- Подвал: версия приложения, состояние связи с базой («сохранено в базе · 13:20», «сохранение...», «нет связи с базой»), переключатель уведомлений. При ошибке сохранения над страницей показываются причина и кнопка «Повторить».
- Заголовок экрана компактный: название и пояснение в одну строку, действия справа; ниже — строка фильтров (подпись и значение в одну строку, значение — выпадающий список). Заголовок вкладки браузера: `<раздел> · Контроль задач`; в «Переписке» — со счётчиком непрочитанных (2) ...
- Подсказки при наведении и при фокусе клавиатурой — собственные, не браузерные.

4.2. Вход

- **Форма.** Логин (Windows-логин или рабочая почта) и пароль; ошибка выводится под полями: «Неверный логин или пароль...». На экране входа — переключатель темы, подсказка с демонстрационными учётными записями и версия приложения.
- **Windows.** При включённом режиме клиент без формы обращается к `POST /api/windows-login`; сервер получает доменного пользователя от IIS (раздел 9.3) и сопоставляет со столбцом «Windows-логин».
- **Резервный вход.** Если включён `allowEmergencyForm`, администратор может войти по форме и в режиме Windows.
- **Защита от самоблокировки.** Форма закрывается, только если прокси действительно передал доменного пользователя в этом же запросе. Пока Windows-вход не отвечает, форма остаётся доступной.

- Демонстрационные учётные записи с паролем `123456` (только четыре, пароли заданы в коде, `src/lib/data.ts`): `user@example.com` (администратор), `petrov@example.com` (руководитель), `sidorov@example.com` и `smirnova@example.com` (исполнители). Остальные сотрудники входят только через Windows.
- Сеанс — подписанный токен на 7 дней; хранится в браузере (`task-control:session`). Ответ 401 завершает сеанс.

4.3. Календарь (`/calendar`)

- Виды: «День», «Неделя» (по умолчанию), «Месяц»; прежнюю «Рабочую неделю» убрали. Выбранный вид запоминается в браузере для каждого сотрудника (`task-control:calendar-view:<id>`). Стрелки листают период, «Сегодня» возвращает к текущей дате. Заголовок и строка фильтров закреплены под шапкой приложения (на ширине от 1024 px); ячейки фильтров имеют минимальную ширину и при нехватке места переносятся на вторую строку, поэтому поиск не наезжает на список категорий.
- Мероприятие — рамка и точка цвета категории; под названием — исполнитель (при нескольких: первый и «+N»; полный список в подсказке). Состояние — полоса у правого края: зелёная (исполнено, ✓), красная (просрочено, красный знак «!», и время), нет полосы (в работе). Исполнено и просрочено взаимоисключающе. Красная линия — текущее время. В «Месяце» до трёх задач в дне и строка «ещё N».
- Создание: нажатие на свободный получасовой интервал или «Новая задача». Открытие: нажатие на задачу (окно карточки, п. 4.9).
- В строке заголовка рядом с названием стоят переключатели «Период» (стрелки, «Сегодня») и «Вид», справа — «Новая задача»; пояснения под названием нет. Строка фильтров: «Исполнитель», «Поиск» (по содержанию) и «Категории» (отметки; «все категории») — в одну строку без наложений.
- **Поиск по содержанию** — по названию, результату, документу и его номеру, коду и названию позиции плана, категории, ФИО исполнителей. Без учёта регистра и «ё/е»; найдутся задачи, где есть все введённые слова (`searchTasks`). Под фильтрами выводится список найденного по всем датам; нажатие переводит календарь к этому дню и открывает задачу. Крестик или Esc очищают поиск.

4.4. Планирование (`/planning`)

- Матрица «раздел планирования × сотрудник» на плановую неделю (пятница — четверг). Занимает высоту окна; шапка с фамилиями и колонка разделов закреплены; прокрутка — внутри; легенда — под матрицей.
- Фильтры: «Плановая неделя» (стрелки, «Текущая»), «Группа» (отделения основного отдела; только руководитель), «Сотрудник» (при выбранной группе — только её состав, первый пункт «Вся группа»; смена группы сбрасывает чужого сотрудника).
- В ячейке — любое число мероприятий (`groupByCell`, сортировка по сроку). Плашка показывает срок, «просрочено» или «✓ исполнено». Если исполнитель отсутствует в день срока, под сроком красным — вид и даты отсутствия. Квадрат перед названием — цвет категории.
- Добавление: кнопка «+ задача» в ячейке (позиция и исполнитель подставляются). Колонки баллов в матрице нет.

- Точка у фамилии — сотрудник отсутствует на этой неделе; цвет — вид отсутствия, подробности в подсказке.
- Исполнитель видит только свой столбец и не отправляет отчёт.
- **Выгрузка:** «Выгрузить CSV» — столбцы `Раздел`; `Позиция`; `Наименование позиции`; `Мероприятие`; `Срок`; `Исполнители`; `Категория`; `Статус`; `Документ`; `Результат`.
- **Документ:** текст по шаблону раздела «Планирование» (по умолчанию «План мероприятий на неделю»); окно с предпросмотром, «Скачать .doc», «Скачать .txt», «Скачать CSV», «Копировать».
- **Направить в отчёт** (руководитель; нужны `reports.view` и `tasks.plan`): формирует отчёт недели из исполненных задач с позицией плана; повторная отправка («Обновить отчёт») заменяет отчёт недели. Статус отчёта виден в строке фильтров.

4.5. Контроль исполнения (`/control`)

- Доска неисполненных задач в шести колонках по сроку: «Просроченные», «Сегодня», «На этой неделе», «На следующей неделе», «В течение месяца», «В течение квартала» (формулы — п. 6.2). Задачи со сроком дальше квартала на доску не попадают — их число показано в сводке «Позже квартала». На широком экране колонки по три в ряд, на узком — одна под другой.
- Фильтры: «Сотрудник» (у исполнителя — только он сам), «Категории», «Сроки» (какие колонки показывать), «Исполнение» («В работе» по умолчанию; «Исполненные» добавляются бледными, с галочкой).
- Сводка над доской; счётчик просрочки в шапке и на колокольчике.
- «+» в колонке создаёт задачу со сроком по умолчанию (п. 6.2); у колонки «Просроченные» кнопки нет.
- Карточка задачи показывает, на сколько дней просрочено, время окончания для «Сегодня»; при отсутствии исполнителя в день срока — красную строку «кто и когда отсутствует».
- «Выгрузить в Word» — текущее содержимое доски: раздел на колонку и таблица (задача, раздел планирования, исполнители, срок, состояние).

4.6. Тетрис (`/tetris`) — график событий отдела

- Месяц: строки — сотрудники (компактные, 40 px), столбцы — дни, полосы — события. Цвет полосы — вид (из словаря «Виды отсутствий»); сплошная заливка — согласовано, диагональная штриховка — заявка, точечный правый край — больничный без даты окончания.
- Метки сроков задач сотрудника на его строке: синяя — назначен, зелёная — исполнено, красная — срок приходится на отсутствие (красная рамка). Нажатие открывает задачу.
- Под именем — остаток отпуска и отгулов на текущий год (исполнитель видит только свой); остаток отпуска ≤ 5 дней выделен. Имя ведёт в карточку.
- Сводка: список видов (фильтр «Виды»), кто отсутствует сегодня (первые трое и «ещё N»), число пересечений, пик отпусков, число заявок на согласование.
- Строка «Отсутствуют, %»: доля сотрудников (отдела или выбранной группы) в согласованном отсутствии любого вида на каждый день; «—» — все на месте; выше 30 % ячейка красная.
- **Пик отпусков** — наибольшая за месяц доля сотрудников в согласованном отпуске (пример: «14% · 1 из 7 · 14–25.09»). Порог — 30 % (ровно 30 % — ещё норма). Превышение: красная цифра, предупреждение с датами, красная полоса в шапке графика. Если порог превысят заявки — то же янтарным. У заявки в списке: «в отпуске будет N% отдела».

- Фильтры: месяц (стрелки, «Текущий»), «Группа», «Виды».
- Нажатие на пустой день строки — новое событие с этой датой; кнопка «Событие» — для себя.
- **Окно события:** сотрудник (руководитель выбирает любого; исполнитель — только себя), вид, даты «с» и «по» включительно, для больничного — «Дата окончания не известна»; при переносе первого дня последний сдвигается (длительность сохраняется); строка расчёта — число дней (отгул — рабочие, остальное — календарные) и для отпуска и отгула остаток «сейчас» и «после»; перечень задач сотрудника со сроком в эти даты; статус (только руководитель; событие руководителя сразу «согласовано»); комментарий до 500 символов. Два события одного сотрудника пересекаться не могут.
- Под графиком: «Пересечения со сроками задач» (метки «Просрочено» / «Под угрозой» / «По заявке», действия «Показать на графике», перенос срока) и «Заявки на согласование» (согласовать/отклонить одной кнопкой).
- **Выгрузка в Excel** (`tetris-ГГГГ-ММ.xlsx` , собственный генератор `src/lib/xlsx.ts` без внешних библиотек), учитывает фильтры «Виды» и «Группа»:
 - «График» — матрица месяца, заливка по виду (насыщенный тон — согласовано, светлее — заявка), отметки сроков (• назначен, ✓ исполнено, ! пересечение), вторая колонка — остатки либо должность, внизу «Отсутствуют, %» и легенда; закреплены шапка и две первые колонки;
 - «События» — список (сотрудник, вид, период, дни, состояние, основание); отклонённые не выгружаются;
 - «Пересечения» — задача, исполнитель, срок, вид и период события, состояние;
 - «Остатки» — только если остатки доступны пользователю.

4.7. Отчётность (`/reports` , `reports.view`)

- Отчёт об исполненных мероприятиях за плановую неделю; стрелки листают недели, «Прошлая» возвращает к прошлой.
- Таблица на высоту окна: закреплены строка с фамилиями, колонка «Разделы планирования» и строка «Итого баллов». Разделы показываются, только если под ними есть записи (`reportRows`). В ячейке — задача, результат, срок, баллы, документ.
- «Печать» (печать браузера; при печати таблица целиком), «Выгрузить CSV» (`otchet-ГГГГ-ММ-ДД.csv` ; разделитель `;` , UTF-8 с BOM; столбцы `Позиция; Наименование позиции; Исполнитель (ФИО); Задача; Результат; Срок; Документ; Баллы`).
- «Документ» — справка по шаблону раздела «Отчётность» (по умолчанию «Справка об исполнении»); доступна, когда за неделю получен отчёт с исполненными мероприятиями; в окне — также «Скачать CSV».

4.8. Показатели эффективности (`/kpi` , `reports.view`)

- Период: «Всё время», «Месяц», «Квартал» (по умолчанию), «Полугодие», «Год» (отсчёт назад от текущего момента по дате исполнения записи отчёта).
- Итоги: всего баллов, исполнено задач, средний балл на сотрудника, лучший результат.
- «Баллы по сотрудникам» — лидер выделен; имя ведёт в карточку; исключённые из оценки уходят вниз с пометкой «вне оценки».
- «Баллы по направлениям» (если включено правилами «Оценки») — общий и средний балл каждого отделения, число сотрудников в расчёте и исполненных задач.

- «Динамика по неделям» — столбец на неделю: сумма баллов отчёта.
- Правила подсчёта задаются в администрировании (п. 4.12.5) и применяются при показе, поэтому пересчитывают и прошлые периоды.

4.9. Карточка задачи (окно)

Одно окно для всех разделов.

поле	ПРАВИЛО
Название	Обязательное; многострочное, на всю высоту левой колонки; до 300 символов (сервер).
Начало, окончание	Дата и время (шаг 15 мин), ввод с клавиатуры или календарём. Окончание — срок. При переносе начала окончание сдвигается, длительность сохраняется. Окончание раньше начала — сообщение сразу, сохранить нельзя.
Раздел планирования	Позиция недельного плана или «Вне плана» (тогда задача только в календаре, контроле и карточках; предупреждение: не попадёт в матрицу и отчёт).
Категория	Значение справочника или «Иное»; цветной квадрат показывает цвет.
Исполнители	Один или несколько (до 20). Выбранные сверху с крестиком; ниже список по подразделениям с «выбрать группу»; при более чем восьми сотрудниках — поиск по фамилии, имени, должности. Пометка отсутствия рядом с фамилией. Каждый исполнитель получает полные баллы.
Документ, номер	Основание/результат (приказ, протокол, ТЗ); до 300 и 100 символов; выводятся в отчёте и документах.
Результат исполнения	До 4000 символов; отметка «Исполнено» доступна только при непустом результате.
Баллы (руководитель)	По умолчанию — базовый вес позиции; значение 0,1–15; пометка «изменено» и «Сбросить к базовому». У задачи вне плана баллов нет.

- Предупреждение «Срок приходится на отсутствие исполнителя» с видом и датами.
- «Сохранить»; «Отмена», крестик, Esc или клик по фону закрывают без сохранения. «Удалить» → «Точно удалить?» (только `tasks.delete`).
- Чужую задачу исполнитель видит только для чтения (кнопка «Закрыть»).

4.10. Сотрудники (`/employees` , `/employees/:id`)

- У руководителя слева — структура отдела по подразделениям с числом задач в работе и просроченных; закреплена при прокрутке. Точка у фамилии — сотрудник сегодня отсутствует (цвет — вид).
- У исполнителя раздел называется «Моя карточка» и показывает только его.
- Карточка: сводка, запланированные и исполненные задачи с баллами; контекст баллов (`employeeScoreContext` : баллы сотрудника, общий и средний по отделу, «относительно среднего» и «вклад» в процентах, признак «вне оценки»); блок «Отсутствия» — остатки на текущий год и список; руководитель здесь же меняет нормы (право на отпуск, перенесённые дни, накопленные отгулы).

4.11. Переписка (/chat)

- Общий канал отдела; сообщения сгруппированы по дням, свои подписаны «(Вы)». Enter — отправить, Shift+Enter — перенос строки. Длина — до 1000 символов.
- Непрочитанные: синий счётчик у пункта меню (>99 — «99+») и на вкладке браузера; при открытии ленты прокручивается к линии «Новые сообщения · N», у новых — точка. Открытая и видимая переписка сразу считается прочитанной. Собственное сообщение отмечает прочитанным всё, что было до него. Отметка у каждого сотрудника своя (chatReads), одинакова на всех его компьютерах; пока чат не открывали, новыми считаются все сообщения коллег.
- Автообновление ленты — каждые 15 секунд.

4.12. Администрирование (/admin)

Раздел открывается, если у пользователя есть хоть одна вкладка. Все изменения сохраняются в базе (кроме «Редактирования UI») и действуют для всех сразу.

4.12.1. Пользователи (users.manage)

- Поля: ФИО, должность, рабочая почта, Windows-логин, подразделение, роль, признак «активен».
- «Редактировать сотрудника», «Добавить сотрудника» (вручную или поиском в Active Directory по фамилии, если AD настроен), удаление.
- Ограничения: Windows-логин и почта непусты и уникальны (без учёта регистра); ФИО непусто; подразделение должно существовать; нельзя отключить/удалить последнего администратора и собственную запись. Отключённый пользователь войти не может.
- Сотрудники приложения — это пользователи базы: добавленный или переведённый сотрудник сразу появляется во всех разделах и фильтрах (syncStaff).

4.12.2. Подразделения (units.manage)

- Дерево «организация → управление → отдел → отделение» со сворачиванием; «+» добавляет вложенное подразделение следующего уровня. Родитель — строго уровнем выше; уровень нельзя менять у подразделения с вложенными; удалить можно только пустое (без вложенных и без сотрудников).
- Основной отдел — u-dept-dev (MAIN_DEPARTMENT_ID); при его отсутствии берётся первый отдел. Отделения основного отдела — группы в «Планировании», «Тетрисе» и выборе исполнителей; сотрудники вне отделений попадают в группу «Без отделения».

4.12.3. Роли и разрешения (roles.manage)

Слева роли с числом разрешений, справа флажки разрешений выбранной роли по группам. Допустимы только ключи из справочника разрешений (п. 3.2). Изменение пересчитывает права вошедшего пользователя.

4.12.4. Аутентификация (authentication.manage)

- Способ: «Форма входа» или «Windows»; флажок резервного входа администратора.
- Переключатель доступен всегда, режим «Windows» можно включить заранее: пока сервер не настроен, вход идёт по форме. Если включить Windows без резервного входа при ненастроенном сервере, сервер отклоняет сохранение (409).

- «Проверить настройку» (`GET /api/windows-check`) показывает, что сервер видит в этом запросе: включено ли доверие прокси, совпал ли секрет, какие заголовки пришли, какой логин взят, с каким сотрудником сопоставлен, и конкретную причину неполадки.

4.12.5. Оценка (`scoring.manage`)

- «Вне общей оценки»: флажки по подразделениям (с вложенными) и поимённо; исключённый подразделением отдельно не снимается.
- «Средний балл» делится: на всех сотрудников (`staff`), только действующих (`active`) или только имеющих баллы за период (`withScore`).
- «Направления»: показывать ли разрез по отделениям.

4.12.6. Редактирование UI (`appearance.manage`)

Личное оформление, хранится в браузере и на сервер не уходит (п. 11.3).

4.12.7. Словари (`dictionaries.manage`)

- Три словаря: «Категории задач» (`taskCategory`), «Виды отсутствий» (`absenceType`), «Статусы задач» (`taskStatus`). Значение: код (до 48 символов, пробелы заменяются дефисами; уникален в словаре без учёта регистра), название (до 160), цвет (`#RRGGBB`) (только у категорий и видов; палитра из 12 цветов или код).
- Добавленная категория сразу появляется в фильтрах «Календаря» и «Контроля» и в окне задачи; без цвета показывается цветом «Иного». Виды отсутствий в модели фиксированы (`vacation` , `trip` , `dayoff` , `sick` , `study`): словарь меняет название и цвет.
- В тёмной теме цвета автоматически высветляются.

4.12.8. Разделы планирования (`planRows.manage`)

- Дерево: разделы и рабочие позиции; сворачивание, «Свернуть/Развернуть всё». Позиция: код (`/^[pL]{pN}+(?:[.-][pL]{pN})+$/u` , до 48 символов), название (до 200), тип (раздел/позиция), базовый вес (0,1–15; у раздела 0).
- Окно: родитель (раздел или «Верхний уровень») + номер → код собирается сам (`2 + 3 → 2.3`).
- Правила иерархии: родитель — существующий раздел; позицию нельзя вложить в себя или потомка; раздел с вложенными нельзя сделать позицией; позицию с задачами нельзя сделать разделом; позицию с вложенными нельзя; коды уникальны без учёта регистра. При смене кода вложенные позиции и связанные задачи переезжают вместе. Удалить можно только позицию без задач и без вложенных.
- Порядок — иерархический по номерам (3.1.2 раньше 3.1.10).

4.12.9. Шаблоны документов (`templates.manage`)

- Шаблон: название (до 160), раздел («Планирование» или «Отчётность»), текст (до 20 000). Шаблон с ошибкой разбора не сохраняется. Редактор: слева текст, справа предпросмотр (для «Планирования» — на мероприятиях текущей недели; для «Отчётности» — на последнем отправленном отчёте); список подстановок меняется вместе с разделом.
- Язык шаблона (`src/lib/templates.ts`): `{{имя}}` — значение (для списка — его длина); `{{#список}}...{{/список}}` — повтор блока (для истинного скаляра — один раз); `{{^список}}...{{/список}}` — блок для пустого списка. Внутри блока видны поля элемента и значения уровнем

выше. Строки, где стоит только тег блока, убираются; три и более пустых строки сжимаются до двух. Кода и HTML нет — только текст. Подстановки — в приложении В.

- Шаблоны без раздела (до 5.3.0): «Справка об исполнении» → «Отчётность», остальные → «Планирование».

4.13. Информация (/info)

Руководство внутри приложения (оглавление слева) и «Скачать PDF»; документ собирается из docs/manual.html, подставляются {{VERSION}} , {{TESTS}} и снимки экрана (в светлой теме показываются светлые варианты). Если версия документации отличается от версии приложения — подпись «Документация к версии X (приложение — Y)». Нажатие на рисунок открывает его крупно; клик переключает «по размеру окна»/натуральную величину; стрелки листают. Кнопка «Библиотека компонентов» ведёт на /kit.

4.14. Библиотека компонентов (/kit)

Витрина собственных компонентов src/kit/ : кнопки, поля, поле поиска, выпадающий список (Select), множественный выбор (MultiSelect), календарь даты и времени (DatePicker), панель выбора цвета (ColorPicker), ползунок (Slider), флажки, переключатели, подсказки (Tooltip), окно (Dialog), макеты.

4.15. Восстановление демонстрационных данных

Действие reset (разрешение data.reset, у администратора): заменяет данные демонстрационным набором (createSeed : 40+ сотрудников в четырёх отделениях, задачи относительно текущей даты, отчёты, переписка, отсутствия и нормы). Сеанс вошедшего сохраняется, если его учётная запись есть в наборе.

05 Модель данных

Версия состояния — 5. Типы — src/lib/types.ts.

5.1. Сущности

Task — задача.

ПОЛЕ	ТИП	ПРИМЕЧАНИЕ
id	string	task- <время> - <счётчик> <случайные символы>
title	string	≤ 300
rowId	string null	код позиции плана
category	string null	код категории; null → «Иное»
assigneeIds	number[]	≤ 20, существующие сотрудники
start, end	ISO	end — спок; end ≥ start
docName (≤ 300), docNumber (≤ 100), result (≤ 4000)	string	
done	boolean	
score	number null	null → базовый вес позиции; 0,1–15
doneAt	ISO null	ставится при первой отметке, при повторном сохранении исполненной задачи сохраняется прежним

Report / ReportEntry — отчёт недели: weekStart (пятница, YYYY-MM-DD), submittedAt, entries[]. Запись: taskId, rowId, rowTitle, assigneeId, title, deadline, result, docName, docNumber, score (замороженный), doneAt. Одна запись на каждого исполнителя.

Message — id, authorId, text (≤ 1000), sentAt. **ChatRead** — employeeId, readAt.

Absence — id, employeeId, type (vacation | trip | dayoff | sick | study), from (YYYY-MM-DD), to (включительно; null — только больничный без даты окончания), status (request | approved | rejected), note (≤ 500), decidedBy, createdAt.

Entitlement — нормы на год: employeeId, year, vacationDays (по умолчанию 28), carriedOver, dayoffAccrued (целые 0–3000).

PlanRow — id (код), title, isHeader, baseScore.

DictionaryEntry — id, dictionary (taskCategory | absenceType | taskStatus), code, title, color?.

ManagedUser — `employeeId` (целое ≥ 1), `email`, `fullName`, `position`, `windowsLogin`, `role`, `active`, `unitId?`.

RoleDefinition — `role`, `name`, `permissions[]`.

Unit — `id`, `parentId`, `kind` (`organization` | `directorate` | `department` | `section`), `name`.

DocumentTemplate — `id`, `name`, `body`, `scope` (`planning` | `reports`).

AuthenticationSettings — `mode` (`form` | `windows`), `allowEmergencyForm`.

ScoringSettings — `excludedUnitIds[]`, `excludedEmployeeIds[]`, `averageBase` (`staff` | `active` | `withScore`), `byDirection`.

Notice (только на сервере) — `id`, `at`, `to[]`, `title`, `body`, `url`; хранится неделю.

AppearanceSettings (только в браузере) — цвета `light` / `dark` (имя переменной CSS без `--` → `#RRGGBB`), `presets[]` (≤ 24), `fontBody`, `fontMono`, `sizes` (`base`, `heading`, `caps`, `subtitle`).

AppState = `Data` без служебных полей + `user` + `appearance`. `Data` — то, что хранится на сервере; `fromData` / `toData` преобразуют одно в другое. Сведения о сеансе и оформлении в базу не попадают.

5.2. Служебные структуры в памяти

`employees`, `employeeById`, `department` (группы), `CATEGORIES` — «живые» списки, обновляемые на месте при загрузке данных (`syncStaff`, `syncCategories`), чтобы экраны и проверки видели актуальный состав без передачи через свойства. Сотрудник строится из пользователя: ФИО делится на фамилию, имя и отчество по пробелам.

5.3. Миграция данных

`isStored` / `migrate` приводят сохранённые данные версий 1–5 к текущей модели: вес строки версии 1 переносится в баллы задач, администратору версий < 4 дописывается `authentication.manage`, отсутствующие подразделения, шаблоны, словари, позиции плана, правила оценки подставляются по умолчанию.

06 Бизнес-правила и расчёты

6.1. Даты

- Начало календарной недели — понедельник; плановой — пятница (`planWeekStart`). Плановая неделя — [пятница, следующая пятница] ; задача относится к неделе по сроку `end` .
- Даты событий — строки `YYYY-MM-DD` без времени и часового пояса; даты задач — ISO.
- Часовой пояс текстов уведомлений и «9:00» напоминания — `Europe/Moscow` (`NOTIFY_TIME_ZONE`), сервер работает в UTC.

6.2. Колонки «Контроля» (`deadlineBucket`)

Пусть `today` — начало сегодняшнего дня, `day` — начало дня срока.

КОЛОНКА	УСЛОВИЕ
<code>overdue</code>	<code>day < today</code>
<code>today</code>	<code>day = today</code>
<code>week</code>	<code>day <</code> ближайшего понедельника после сегодняшнего
<code>nextWeek</code>	<code>day <</code> понедельник + 7 дней
<code>month</code>	<code>day < max(начало следующего месяца, today + 30 дней)</code>
<code>quarter</code>	<code>day < max(конец текущего квартала, today + 3 месяца)</code>
<code>later</code>	иначе (на доску не выводится, считается в сводке)

Срок по умолчанию для новой задачи из колонки (`defaultDeadlineFor` , время 18:00):

`overdue` / `today` — сегодня; `week` — ближайшая пятница, а если она сегодня или прошла — завтра (в пределах недели, иначе сегодня); `nextWeek` — пятница следующей недели; `month` — пятница через две недели; `quarter` / `later` — пятница через семь недель.

Просрочена неисполненная задача, чей срок попадает в `overdue` (определяется по дню, не по времени).

6.3. Баллы

баллы задачи	= score задачи, если задан, иначе <code>baseScore</code> позиции (0 для задачи вне плана)
баллы записи отчёта	= баллы задачи на момент отправки (каждому исполнителю полностью)
баллы сотрудника	= Σ баллов записей отчётов, чья дата исполнения (<code>doneAt</code>) попадает в период
диапазон	= 0,1 ... 15, шаг 0,1 (<code>clampScore</code> округляет до десятых)
базовые веса позиций	= 5 или 2 (в демо-плане)

- Баллы не пересчитываются задним числом: после изменения баллов задачи отчёт недели нужно отправить заново.

- Исполнитель создаёт задачу без индивидуальных баллов; индивидуальные баллы существующей задачи меняет только обладатель `tasks.score` (иначе они сохраняются прежними).

6.4. Построение отчёта (`buildReportEntries`)

Для задач плановой недели: `done = true`, `rowId` указывает на существующую позицию (не раздел); на каждого исполнителя создаётся запись с названием позиции, сроком, результатом, документом, баллами и `doneAt` (при отсутствии — срок). `upsertReport` заменяет отчёт той же недели и сортирует отчёты по неделе.

6.5. Правила оценки

`summarize(totals, ctx)`:

- исключаются сотрудники из `excludedEmployeeIds` и все сотрудники подразделений из `excludedUnitIds` (с вложенными);
- `total` — сумма баллов остальных, округление до десятых;
- `counted` — знаменатель: при `withScore` — те, у кого баллы > 0; при `active` — активные учётные записи; при `staff` — все не исключённые;
- `average = total / counted` (0 при `counted = 0`).

`employeeScoreContext` для карточки: для `staff` знаменатель — переданная численность группы минус исключённые, иначе `counted`; `relativeToAverage = employeeTotal / average × 100`, `contribution = employeeTotal / total × 100` (null при нулях).

`kpiByDirection`: итог каждого отделения считается по его собственным баллам без учёта исключений (иначе у руководства были бы задачи, но нулевые баллы); отделение целиком «вне оценки», если исключены все его сотрудники. Строки сортируются: сначала учитываемые, затем по убыванию баллов.

6.6. Отсутствия

дни отсутствия	= календарные дни; отгул (dayoff) — только рабочие (пн-пт)
больничный без даты	= длится до «сегодня + 3 дня» (OPEN_SICK_DAYS), пока его не закроют
остаток отпуска	= <code>vacationDays + carriedOver</code> — использовано — идёт сейчас — запланировано (заявки в остаток не входят, показываются отдельно)
остаток отгулов	= <code>dayoffAccrued</code> — использовано (заявки отдельно)
пик отпусков	= max по дням месяца (сотрудников в согласованном отпуске ÷ размер группы)
порог	= 30 % (VACATION_LIMIT); превышение — строго больше

- Отпуск классифицируется по сегодняшнему дню: окончился → «использовано», начнётся → «запланировано», иначе → «идёт сейчас».
- Для года считаются только дни, попадающие в этот год.
- Отклонённые отсутствия (`rejected`) в графике, остатках, пересечениях и проверках не участвуют.
- Остатки не хранятся — каждый раз вычисляются из норм и отсутствий.
- Проверка перед сохранением (`validateAbsence`): первый день задан; без даты окончания можно только больничный; последний день не раньше первого; интервал не пересекается с

другим активным событием того же сотрудника.

- Для не-больничного при пустом `to` ставится `to = from`.

6.7. Пересечения (`findClashes`)

Для каждой неисполненной задачи и каждого её исполнителя ищется активное (не отклонённое) отсутствие, покрывающее день срока. Степень: `request` – отсутствие ещё заявка; иначе `overdue` – срок уже прошёл; иначе `risk` – срок впереди. Результат отсортирован по сроку. Используется в календаре, планировании, контроле, окне задачи и «Тетрисе».

6.8. Проверки задачи (`validateTask`)

УСЛОВИЕ	СООБЩЕНИЕ
Пустое название	«Введите название задачи.»
Окончание раньше начала	«Окончание раньше начала: исправьте дату или время.»
Нет исполнителей	«Выберите хотя бы одного исполнителя.»
Отметка без результата	«Чтобы отметить исполнение, опишите результат.»
Баллы вне 0,1–15	«Баллы – число от 0,1 до 15.»

Сервер повторяет проверки и дополнительно: позиция плана существует; категория входит в справочник; исполнители – существующие сотрудники; даты корректны.

6.9. Права на действия (редьюсер)

Каждое действие, у которого у пользователя нет права, возвращает прежнее состояние; сервер трактует это как отказ (HTTP 403). Сводка: `saveTask` / `deleteTask` – п. 3.4; `submitReport` – `reports.view` и `tasks.plan`; `saveAbsence` / `deleteAbsence` – `absenceAccess`; `decideAbsence` – `absences.manage`; `saveEntitlement` – `entitlements.manage`; `saveDictionary` / `deleteDictionary` – `dictionaries.manage`; `savePlanRow` / `deletePlanRow` – `planRows.manage`; `saveManagedUser` / `addManagedUser` / `deleteManagedUser` – `users.manage`; `saveUnit` / `deleteUnit` – `units.manage`; `saveTemplate` / `deleteTemplate` – `templates.manage`; `saveRolePermissions` – `roles.manage`; `saveAuthentication` – `authentication.manage`; `saveScoring` – `scoring.manage`; `reset` – `data.reset`. `sendMessage` и `markChatRead` требуют только входа; `saveAppearance` не требует прав и не сохраняется на сервере.

07 Серверная часть

Единый обработчик `createApi` (`server/app.ts`) работает как функция Vercel, как часть сервера Vite в разработке и как HTTP-сервер iisnode (`server/iis.ts`). Все ответы — JSON, `cache-control: no-store`. Максимальный размер тела запроса — 256 КБ.

7.1. Маршруты

МЕТОД И АДРЕС	НАЗНАЧЕНИЕ	ДОСТУП
GET <code>/api/health</code>	<code>{ ok, storage }</code> ; 503 <code>{ ok:false, error }</code> , если база не отвечает	открыт
GET <code>/api/authentication</code>	<code>{ authentication, windowsAvailable, directoryAvailable }</code> — для экрана входа	открыт
POST <code>/api/login</code>	<code>{ email, password }</code> → <code>{ token, user }</code>	открыт
POST <code>/api/windows-login</code>	вход через Windows → <code>{ token, user }</code>	открыт, 409 если режим не включён, 401/403 — нет доменного пользователя/сопоставления
GET <code>/api/state</code>	<code>{ data }</code> — данные без журнала уведомлений, отметки «прочитано» — только своя	токен
POST <code>/api/action</code>	<code>{ action }</code> → <code>{ data }</code>	токен
GET <code>/api/notices?since=ISO</code>	<code>{ notices, now }</code> — новые уведомления сотрудника (не более 20)	токен
GET <code>/api/windows-check</code>	разбор запроса для проверки Windows-входа + сопоставление	токен, <code>authentication.manage</code>
GET <code>/api/directory-users?q=</code>	поиск пользователей Active Directory (≥ 2 символов, до 20)	токен, <code>users.manage</code>

Токен передаётся заголовком `Authorization: Bearer ...`. Статика и любой неизвестный адрес вне `/api/` отдают `index.html` (одностраничное приложение). Неизвестный адрес API — 404.

7.2. Выполнение действия

- 1 Проверяется токен; восстанавливается пользователь по актуальным данным (`effectiveUser`): отключённый или удалённый — 401.

- 2 Тело разбирается `parseAction`: допустимые типы — `PERSISTED`; структура и длины всех полей проверяются, время подставляется серверное.
- 3 В транзакции хранилища (`repo.update`) состояние до и после сравнивается; если права не хватило — 403 и ничего не сохраняется.
- 4 Для `saveTask`, `sendMessage`, `saveAbsence`, `decideAbsence`, `submitReport` вычисляются события (п. 10) и дописываются в журнал уведомлений; записи старше недели удаляются.
- 5 Клиенту возвращаются актуальные данные.

Ошибки: 400 (неверные данные с описанием поля), 401 (сеанс истёк/учётная запись отключена), 403, 404, 409, 413, 503, 500 («Ошибка сервера. Повторите попытку позже.»; подробности — в журнал).

7.3. Клиент и синхронизация

- Режим `remote` (по умолчанию): изменение сразу применяется на экране (оптимистично), затем отправляется; ответ сервера заменяет локальные данные; при ошибке данные перечитываются, пользователь видит причину и «Повторить».
- Данные перечитываются раз в 60 секунд и при возврате на вкладку; «Переписка» — каждые 15 секунд; уведомления — каждые 20 секунд (таймер в отдельном потоке, не замедляется в фоновой вкладке).
- Режим `local` (тесты, `VITE_BACKEND=local`): данные в `localStorage` (`task-control:v...`), вход проверяется локально.

7.4. Демо-данные и пустая база

Пустая база при первом обращении заполняется `createSeed`. Сотрудники в демо — `src/lib/staff.ts` (40 человек отдела разработки в четырёх отделениях) плюс сотрудники 1–7.

08 Хранилища данных

Выбор по переменным окружения (`server/storage.ts`), в порядке приоритета:

- 1 **Microsoft SQL Server** — если задан `MSSQL_SERVER` (приоритетнее `DATABASE_URL`).
- 2 **PostgreSQL** — `DATABASE_URL` или `POSTGRES_URL` (демо-сайт Vercel, Neon).
- 3 **Файл** — `data\db.json` рядом с сервером IIS либо `.data/dev-db.json` в разработке (`TC_DB_FILE`); без файла и настроек — память.

Интерфейс `Repo`: `kind`, `read()`, `update(fn)` (изменение под блокировкой, откат при исключении), `ping()`, `close()`.

8.1. SQL Server (`server/mssqlRepo.ts`)

- Версия — 2016 и новее (проверено на 2025). Драйвер `mssql`; `encrypt` и `trustServerCertificate` по умолчанию включены (самоподписанные сертификаты в сети); настройки: `MSSQL_SERVER` (допускается `СЕРВЕР\ЭКЗЕМПЛЯР`), `MSSQL_INSTANCE`, `MSSQL_PORT`, `MSSQL_DATABASE` (по умолчанию `TaskControl`), `MSSQL_USER`, `MSSQL_PASSWORD`, `MSSQL_DOMAIN`, `MSSQL_ENCRYPT`, `MSSQL_TRUST_SERVER_CERTIFICATE`.
- **Схема создаётся сама** при первом обращении отдельными операторами (каждый защищён проверкой существования; новые столбцы дописываются к существующим таблицам): `tc_meta`, `tc_tasks`, `tc_absences`, `tc_entitlements`, `tc_reports`, `tc_messages`, `tc_chat_reads`, `tc_users`, `tc_roles`, `tc_units`, `tc_dictionaries`, `tc_plan_rows`, `tc_templates`, `tc_notices`.
- Ограничения целостности: `CHECK` на вид и статус отсутствия; составной ключ норм (`employee_id`, `year`); уникальные индексы на почту, Windows-логин (отфильтрованный, непустой) и (`dictionary`, `code`); индексы по `tc_absences(employee_id, date_from)` и `tc_notices(at)`. Массивы и объекты хранятся JSON-строкой.
- **Конкурентность**: изменения выполняются в транзакции под общей блокировкой `sp_getapplock` (исключительная, таймаут 15 с); записываются только добавленные, изменённые и удалённые строки.
- `tc_meta` хранит служебные значения, включая отметку версии прав.
- Отдельная утилита `scripts/setup-sql.ps1` создаёт базу и учётную запись приложения (`tc_app`, случайный пароль) и вписывает настройки и `AUTH_SECRET` в `web.config`; параметр `-Reset` задаёт новый пароль.
- **Скрипт для ручного запуска** `scripts/create-database.sql` (SSMS или `sqlcmd`) создаёт базу `TaskControl`, учётную запись `tc_app` (пароль задаётся в файле; пока не заменён, учётная запись не создаётся), права `db_owner` и все 14 таблиц с индексами; безопасен при повторном запуске. Строится командой `npm run build:sql` из `SCHEMA` (`server/databaseScript.ts`); тест сверяет файл со схемой. Таблицы создаются пустыми, при первом открытии приложения пустая база заполняется демонстрационными данными. После ручного создания базы публикуют с ключом `/nosql`, а настройки `MSSQL_*` и `AUTH_SECRET` вписывают в `web.config`.

8.2. PostgreSQL (`server/pgRepo.ts`)

Аналогичная схема и семантика (транзакция, блокировка, запись изменившихся строк). В тестах используется встроенный PGlite.

09 Аутентификация и безопасность

9.1. Сеанс

Токен `payload.signature`: `payload` — base64url JSON (`email`, `employeeId`, `role`, `exp`), подпись HMAC-SHA256, срок 7 дней. Проверка подписи — `timingSafeEqual`. Секрет — `AUTH_SECRET`, а если не задан — SHA-256 от настроек базы; без базы — `task-control-dev-secret` (только разработка). В токене нет разрешений: права пересчитываются сервером из актуальных ролей при каждом запросе.

9.2. Форма

Логин сопоставляется с Windows-логинем или почтой без учёта регистра; пароль сверяется со списком демонстрационных учётных записей (`accounts`); пользователь должен быть активным.

9.3. Windows (бесшовный вход)

Схема: браузер → IIS с Windows Authentication (Negotiate/NTLM) → локальный Node.js.

Приложение пароль не запрашивает и `WWW-Authenticate` не отправляет.

- Доменного пользователя подтверждает IIS и передаёт заголовком. Просматриваются заголовки в порядке: `WINDOWS_AUTH_HEADER` (если задан), затем `x-windows-user`, `x-iisnode-logon_user`, `x-iisnode-auth_user`, `x-iis-user`, `x-remote-user`, `x-forwarded-user`, `remote-user`, `auth-user`.
- Доверие включается **только** `WINDOWS_AUTH_TRUST_PROXY=true`; иначе заголовок мог бы подделать любой, кто доберётся до порта.
- Необязательный общий секрет прокси `WINDOWS_AUTH_PROXY_SECRET`: запрос должен нести его в заголовке `x-windows-auth-secret`.
- Логин нормализуется (`DOMAIN\ivanov`, `ivanov@corp.local`, `ivanov` → сопоставление по всем трём видам); ищется активный пользователь с таким Windows-логинем.
- `deploy-iis.bat` настраивает правило перезаписи, которое кладёт `{LOGON_USER}` в серверную переменную `HTTP_X_WINDOWS_USER`.
- Поиск в Active Directory (необязательный): `AD_SEARCH_ENABLED=true`, модуль RSAT ActiveDirectory через PowerShell (`AD_POWERSHELL_PATH`, `AD_SEARCH_BASE`); без LDAP-библиотек и интернета; таймаут 8 с; вводимая строка очищается от спецсимволов.

9.4. Защита данных и прочее

- Все входящие данные проверяются на сервере независимо от клиента; редьюсер доверяет только проверенной форме.
- Клиенту не отдаются чужие отметки «прочитано» и журнал уведомлений; свои уведомления — через `/api/notices`.
- Статика отдаётся только из папки `public` (выход за её пределы запрещён); `web.config` скрывает папки `node_modules`, `iisnode`, `data`.

- Идентификаторы и тексты экранируются средствами React; параметры SQL — через типизированные параметры драйвера.
- Приложение не загружает ничего из интернета: шрифты — только из списка системных.

10 Уведомления

Сервер сравнивает данные до и после действия (`detectEvents`) и записывает уведомления в журнал; открытая вкладка раз в 20 секунд запрашивает новые и показывает их средствами Chrome (Notification API), а `public/sw.js` по нажатию открывает нужный раздел в уже открытой вкладке (`postMessage` типа `navigate`) или новую. Внешних служб нет. Требуется защищённый адрес (HTTPS) — требование браузера.

СОБЫТИЕ	КОМУ	РАЗДЕЛ ПО НАЖАТИЮ
Новая задача / назначение	Добавленным исполнителям (кроме автора)	<code>/control</code>
Изменён срок	Остальным исполнителям (кроме автора), задача не исполнена	<code>/control</code>
Задача исполнена	Обладателям <code>tasks.plan</code> (кроме автора)	<code>/control</code>
Новая заявка на отсутствие	Обладателям <code>absences.manage</code> (кроме автора)	<code>/tetris</code>
Заявка согласована / отклонена	Автору заявки	<code>/tetris</code>
Отчёт недели направлен	Обладателям <code>reports.view</code>	<code>/reports</code>
Начислены баллы	Каждому исполнителю отчёта	<code>/employees</code>
Сообщение в переписке	Всем активным, кроме автора	<code>/chat</code>
Просроченные задачи	Каждому исполнителю раз в сутки, при первом обращении после 9:00 по Москве; до трёх задач и «...и ещё N»	<code>/control</code>

Журнал хранится неделю. Напоминание о просрочке создаёт сам сервер (идентификатор `ntc-overdue-<сотрудник>-<день>` исключает повтор), внешний планировщик не нужен. После закрытой вкладки при следующем открытии показываются последние три пропущенных уведомления и строка «И ещё уведомлений: N». Включение — приглашение над страницей «Включите уведомления Chrome» либо переключатель в подвале; состояние запоминается в браузере (`task-control:notices` , `...-since`).

11 Интерфейс и оформление

11.1. Собственная библиотека компонентов

Стандартные элементы браузера не используются: выпадающие списки, календарь даты и времени, флажки, переключатели, поля, подсказки, окна, ползунки, панель цвета и полосы прокрутки оформлены в `src/kit/` единой темой. Полосы прокрутки — через `::-webkit-scrollbar` (в Chrome 121+ свойства `scrollbar-width` / `scrollbar-color` отключили бы их), для Firefox запасной блок `@supports not selector::-webkit-scrollbar`. Тест `scrollbars.test.ts` не даёт вернуть системные свойства.

11.2. Темы

Две темы — тёмная (основная, «тёплый графит») и светлая; выбор запоминается в браузере отдельно для каждого сотрудника (`task-control:theme:<id>`); общий ключ `task-control:theme` хранит последнюю тему компьютера и берётся для нового сотрудника. Переключатель есть в шапке и на экране входа.

11.3. Редактирование UI (личное оформление)

Хранится в `localStorage` по ключу `task-control:appearance:<id>` и на сервер не уходит; сохраняются только отличия от дизайн-системы.

- **Шрифты:** основной (7 вариантов: Segoe UI, Golos Text, Arial, Tahoma, Verdana, Georgia, Times New Roman) и для чисел и дат (Cascadia Mono, Consolas, IBM Plex Mono, Courier New). Значения вне списка отбрасываются.
- **Размеры текста** ползунком (шаг 0,5 px) с живым образцом: основной 12–20 (14), заголовки 14–24 (16), подписи 9–14 (11), пояснения 11–18 (14).
- **Цвета** — восемь групп (основа, линии и подложки, акцент, состояния, календарь, категории задач, виды событий, логотип); светлая и тёмная правятся отдельно; переключатель «Светлая / Тёмная» над списком включает тему во всём приложении. Панель выбора цвета: поле насыщенности и яркости, полоса тона, палитра приложения, код `#RRGGBB`. Токены со спутником `-rgb` пересчитываются автоматически. «По умолчанию» и «Вернуть всё по умолчанию».
- **Готовые цветовые темы:** светлые — «Тёплый мел» (исходная), «Песочная», «Северная», «Хвойная»; тёмные — «Ночной мел» (исходная), «Индиго», «Уголь», «Тайга». Тема задаёт поверхности и акцент; цвет текста не меняется (контраст сохраняется). Свои темы — до 24 штук.
- **Логотип и значок вкладки** следуют за акцентом: один цвет, часть кубиков с прозрачностью 55 %; явно заданные цвета логотипа главнее и сплошные. Значок вкладки — SVG (`data:`), строится из тех же цветов.
- Цвета отдельных значений словарей главнее цвета по умолчанию, заданного здесь.

11.4. Адаптивность

Проверяются ширины 375, 768, 1024 и 1440 px: нет горизонтальной прокрутки страницы, кнопки на телефоне не меньше 44 px, поля ввода крупнее (телефон не увеличивает страницу). На телефоне шапка компактная (знак и кнопки в одной строке, разделы — во

второй, с прокруткой вбок), фильтры встают столбцом, колонки «Контроля» — одна под другой; календарь и матрицы прокручиваются внутри себя.

11.5. Доступность и прочее

Фокус клавиатуры виден; подсказки доступны с клавиатуры (Tab); списки управляются стрелками, Enter, Esc; учитывается `prefers-reduced-motion`; язык интерфейса — русский (`lang="ru"`), даты и числа форматируются по `ru-RU`; склонение «балл/балла/баллов» (`plural`).

12 Нефункциональные требования

ОБЛАСТЬ	ТРЕБОВАНИЕ
Сеть	Работа без обращений в интернет; ни один ресурс не загружается извне.
Браузеры	Chrome и Edge 109 и новее (сборка <code>chrome109</code> , <code>edge109</code> , <code>firefox115</code> , <code>safari115.6</code>); не использовать возможности новее Chrome 109 (например, <code>color-mix()</code>).
Сервер	Node.js 18+ на сервере IIS (на разработке 20.19+, проверено на 24.13); IIS с модулями iisnode и URL Rewrite.
База	SQL Server 2016+; одновременные правки разных сотрудников не теряются (блокировка + транзакции).
Права	Проверка на сервере независимо от интерфейса; один источник правил для клиента и сервера.
Целостность	Любое действие либо сохраняется целиком, либо не сохраняется.
Отзывчивость	Оптимистичное обновление интерфейса; данные у коллег — до минуты (чат — до 15 с, уведомления — до 20 с).
Размеры	Тело запроса ≤ 256 КБ; длины полей ограничены (см. модель данных).
Отказоустойчивость	<code>/api/health</code> проверяет само соединение с базой; недоступная база даёт 503, а не «белый экран»; ошибка сохранения видна пользователю с кнопкой «Повторить».
Единый интерфейс	Только собственная библиотека компонентов, без стандартных элементов браузера и стандартных полос прокрутки.
Версионирование	Семантическое: каждое изменение — новая версия, запись в <code>CHANGELOG.md</code> , версия в <code>package.json</code> , README, подвале, экране входа и документации.

13 Развёртывание

13.1. Переменные окружения

ПЕРЕМЕННАЯ	НАЗНАЧЕНИЕ
<code>MSSQL_SERVER</code> , <code>MSSQL_DATABASE</code> , <code>MSSQL_USER</code> , <code>MSSQL_PASSWORD</code> , <code>MSSQL_DOMAIN</code> , <code>MSSQL_INSTANCE</code> , <code>MSSQL_PORT</code> , <code>MSSQL_ENCRYPT</code> , <code>MSSQL_TRUST_SERVER_CERTIFICATE</code>	SQL Server
<code>DATABASE_URL</code> / <code>POSTGRES_URL</code>	PostgreSQL; используется, если <code>MSSQL_SERVER</code> пуст
<code>AUTH_SECRET</code>	ключ подписи сеансов
<code>WINDOWS_AUTH_TRUST_PROXY</code> (<code>true</code>), <code>WINDOWS_AUTH_HEADER</code> , <code>WINDOWS_AUTH_PROXY_SECRET</code>	вход через Windows
<code>AD_SEARCH_ENABLED</code> , <code>AD_SEARCH_BASE</code> , <code>AD_POWERSHELL_PATH</code>	поиск в Active Directory
<code>PORT</code>	порт (iisnode задаёт сам)
<code>TC_STATIC_DIR</code> , <code>TC_DB_FILE</code>	папка статики и файл данных сервера
<code>VITE_BACKEND=local</code>	клиент без сервера (на этапе сборки)

В IIS настройки лежат в разделе `appSettings` файла `web.config`.

13.2. Корпоративная сеть: IIS + iisnode + SQL Server

Адрес по умолчанию — `http://10.199.127.27:1500/`, сайт и пул — `PLAN`, папка — `C:\inetpub\wwwroot\PLAN` (задаются переменными в первых строках `deploy-iis.bat`).

Состав комплекта `dist-iis`: `server.cjs` (сервер, собран esbuild), `public\` (интерфейс и руководство), `web.config`, `setup-sql.ps1`, `data\`, `version.txt`, `README.txt`. Папка в репозитории не хранится — это результат `npm run build:iis`.

Сервер без интернета — ничего не скачивается и не собирается на нём:

- 1 На компьютере с интернетом: `npm install`, `npm run pack:iis` → `release\task-control-iis-<версия>.zip` (комплект + `deploy-iis.bat` + `deploy-iis-remote.bat` + `scripts/` + инструкция + `README.txt`). Тот же архив приложен к каждому выпуску на GitHub.
- 2 На сервер носителем переносятся: архив, установщики Node.js 18+, iisnode и URL Rewrite (`.msi`). Роль «Веб-сервер IIS» ставится из компонентов Windows.
- 3 Архив распаковывается, при необходимости правятся `IP`, `PORT`, `SQLSERVER` в начале `deploy-iis.bat`, затем запускается `deploy-iis.bat` от имени администратора.
- 4 Проверка: `http://<адрес>:<порт>/api/health` → `{"ok":true,"storage":"sqlserver"}`.

Что делает `deploy-iis.bat` (ключи `/nobuild`, `/nosql`, `/resetsql`):

- 1 Проверяет права администратора, `appcmd.exe`, `node.exe`, `iisnode`, URL Rewrite.
- 2 Определяет комплект: рядом `node_modules` → собирает (`npm run build:iis`; `/nobuild` пропускает); иначе берёт готовый `dist-iis`; иначе объясняет, как получить архив.
- 3 Показывает версию комплекта и версию на сервере, останавливает пул, копирует файлы (`robocopy /MIR`, без перезаписи `web.config`, `data`, `iisnode`); при первой публикации создаёт `web.config` и записывает путь к `node.exe`.
- 4 Выдаёт права (`IIS_IUSRS` — чтение, запись в `data` и `iisnode`; то же для учётной записи пула).
- 5 Создает пул `PLAN` (без управляемого кода, `AlwaysRunning`, без простоя и перезапусков по расписанию).
- 6 Создает сайт `PLAN` с привязкой `http/<IP>:<PORT>`.
- 7 Настраивает Windows-аутентификацию (анонимная и обычная выключены, `Negotiate` выше `NTLM`) и правило, передающее `LOGON_USER`.
- 8 Настраивает SQL Server (`setup-sql.ps1`): учётная запись `tc_app` со случайным паролем, база `TaskControl`, владелец — эта учётная запись; пишет настройки и `AUTH_SECRET` в `web.config`. Нужны права `sysadmin` (или `securityadmin` и `dbcreator`) и смешанный режим проверки подлинности в SQL Server.
- 9 Открывает порт в брандмауэре, запускает пул и сайт и проверяет, что на сервере версия комплекта (иначе сообщает, что файлы не обновились).

Обновление: привезти новый архив и снова запустить `deploy-iis.bat`; `web.config`, `data` и журналы `iisnode` сохраняются.

Публикация с другого компьютера сети (`deploy-iis-remote.bat -Server ИМЯ`, `scripts/deploy-remote.ps1`): комплект (`dist-iis` и `deploy-iis.bat`) упаковывается, по PowerShell Remoting (WinRM) передаётся на сервер, распаковывается во временную папку, и там выполняется тот же `deploy-iis.bat` (настройки передаются переменными окружения `TC_IIS_SITE`, `TC_IIS_POOL`, `TC_IIS_IP`, `TC_IIS_PORT`, `TC_IIS_TARGET`, `TC_SQLSERVER`, `TC_SQLDB`, `TC_SQLLOGIN`). Скрипт предварительно проверяет сервер (права администратора, роль IIS, `iisnode`, URL Rewrite, Node.js, установленная версия), выводит сообщения публикации, сохраняет их в `deploy-remote-<сервер>.log`, показывает установленную версию и проверяет `/api/health`. Параметры: `-Server`, `-Check` (только проверить сервер), `-AskCredential`, `-NoSql`, `-ResetSql`, `-Site`, `-Pool`, `-Ip`, `-Port`, `-Target`, `-SqlServer`, `-SqlDb`, `-SqlLogin`, `-Build`. Требования: права администратора на сервере, включённый WinRM (`Enable-PSRemoting -Force`), порт 5985. SQL Server на третьем компьютере: учётные данные с сервера IIS дальше не передаются, поэтому базу создают скриптом `create-database.sql` и публикуют с `-NoSql`.

Журналы ошибок: `C:\inetpub\wwwroot\PLAN\iisnode\*.txt` (вывод Node.js), журнал Windows «Приложение», вывод батника (`deploy-iis.bat > deploy-log.txt 2>&1`).

`web.config` комплекта: обработчик `iisnode` для `server.cjs`; правило перезаписи всех адресов на `server.cjs`; скрытые сегменты `node_modules`, `iisnode`, `data`; `httpErrors existingResponse="PassThrough"`; `iisnode nodeProcessCountPerApplication="1"`, `loggingEnabled="true"`, `watchedFiles="web.config;*.cjs"`.

После публикации: открыть адрес, войти администратором, проверить Windows-вход («Администрирование → Аутентификация → Проверить настройку»), заполнить Windows-логины сотрудников и только потом включить режим «Windows-аутентификация».

13.3. Демонстрационный стенд (Vercel)

`npm run build:vercel` (Build Output API v3: статика из `dist/` и функции `/api/login`, `/api/windows-login`, `/api/authentication`, `/api/directory-users`, `/api/state`, `/api/action`, `/api/health`, `/api/notices`, `/api/windows-check`, runtime `nodejs22.x`, `maxDuration 30 c`) → `npm run deploy` (публикация из временной копии без `.git`). База — Neon PostgreSQL (`DATABASE_URL`). Кэш: `/assets/* — immutable`, `sw.js — no-cache`.

14 Разработка, проверки и версионирование

Стек: Vite 8, React 19, React Router 8, TypeScript 6 (строгий режим; проверка `tsc -b`, не `--noEmit`), `oxlint`, Vitest 5 + Testing Library + `jsdom`, `puppeteer-core` (снимки экрана), драйверы `mssql` и `pg`, PGlite для тестов PostgreSQL.

Команды:

КОМАНДА	ЧТО ДЕЛАЕТ
<code>npm run dev</code>	разработка вместе с API (<code>http://localhost:5173</code>), данные в <code>.data/dev-db.json</code>)
<code>npm run build</code>	проверка типов и сборка в <code>dist/</code>
<code>npm run build:vercel</code> / <code>npm run deploy</code>	сборка и публикация на Vercel
<code>npm run build:iis</code>	комплект <code>dist-iis/</code>
<code>npm run build:sql</code>	скрипт создания базы <code>scripts/create-database.sql</code> из схемы приложения
<code>npm run pack:iis</code>	архив <code>release/task-control-iis-<версия>.zip</code> (Windows)
<code>deploy-iis-remote.bat -Server ИМЯ</code>	публикация в IIS с другого компьютера сети
<code>npm test</code>	модульные, интерфейсные и серверные тесты
<code>npm run test:mssql</code>	тесты хранилища SQL Server (<code>MSSQL_TEST_SERVER</code> , <code>MSSQL_TEST_USER</code> , <code>MSSQL_TEST_PASSWORD</code>)
<code>npm run lint</code>	статический анализ
<code>npm run docs</code>	тесты, сборка, дымовая проверка в Chrome на 4 ширинах, снимки в двух темах, PDF-руководство

Структура проекта: `src/screens/` — разделы; `src/components/` — составные компоненты; `src/kit/` — библиотека компонентов; `src/lib/` — предметная логика без React (редьюсер, права, расчёты, даты, шаблоны, загрузки, `xlsx`, оформление); `server/` — API и хранилища; `scripts/` — сборки Vercel и IIS, упаковка, `setup-sql.ps1`; `docs/` — руководство и инструкции; `public/` — статика и `sw.js`.

Проверки. На 5.27.2: 22 файла тестов, 285 пройдено и 12 пропущено (тесты SQL Server требуют сервера); `tsc -b` и `oxlint` без замечаний. Покрыты: логика и расчёты, права, редьюсер, шаблоны, оформление и темы, выбор цвета и ползунки, загрузки (Excel «Тетриса»), уведомления, API, хранилища PostgreSQL и SQL Server, Windows-вход, полосы прокрутки, информация.

Версионирование. `MAJOR.MINOR.PATCH`: MAJOR — смена модели данных, ролей или облика; MINOR — новые возможности; PATCH — исправления. Источник — `package.json`; значение попадает в интерфейс (`__APP_VERSION__`), README, документацию. История —

`CHANGELOG.md`. Выпуск: версия → CHANGELOG → проверки → Vercel → коммит, тег `vX.Y.Z`, GitHub Release (с архивом для IIS). PDF-руководство пересобирается по отдельной просьбе.

15 Известные ограничения

- 1 **Пароли демонстрационных учётных записей** заданы в коде (`123456`, четыре записи); у остальных сотрудников пароля нет. Для рабочей эксплуатации используется только вход через Windows. Форма остаётся как резервный вход администратора.
- 2 **Не мгновенная синхронизация.** Изменения коллег подгружаются раз в минуту и при возврате на вкладку (чат — 15 с, уведомления — 20 с); push-канала нет.
- 3 **Шаблоны документов** — простой текст: в `.doc` один шрифт, без таблиц и выделения.
- 4 **Оформление личное** и хранится в браузере: при смене браузера или очистке данных сайта оно сбрасывается; на сервере его нет.
- 5 **Уведомления Chrome** требуют HTTPS и разрешения браузера; вкладка приложения должна быть открыта (можно закреплённой, в фоне). Показывает их Windows: режим «Не беспокоить» их скрывает.
- 6 **Сеанс** действует 7 дней и не отзывается централизованно (отключение пользователя делает токен недействительным на следующем запросе, смена `AUTH_SECRET` разлогинивает всех).
- 7 **Баллы не пересчитываются задним числом:** после правки баллов задачи отчёт недели отправляется заново.
- 8 **Виды отсутствий** фиксированы пятью значениями; словарь меняет название и цвет, но не добавляет новые виды.
- 9 **Один процесс Node.js** (`nodeProcessCountPerApplication="1"`): in-memory списки сотрудников и категорий общие для запросов процесса.
- 10 Выгрузки (`.xlsx`, `.doc`, `.csv`) формируются в браузере; в Excel CSV открывается через «Данные → Из текста/CSV» с разделителем «точка с запятой».

16 Приложения

Приложение А. Права доступа к вкладкам администрирования

ВКЛАДКА	РАЗРЕШЕНИЕ
Пользователи	users.manage
Подразделения	units.manage
Роли и разрешения	roles.manage
Аутентификация	authentication.manage
Оценка	scoring.manage
Редактирование UI	appearance.manage
Словари	dictionaries.manage
Разделы планирования	planRows.manage
Шаблоны документов	templates.manage

`canOpenAdmin` — истина, если у пользователя есть `admin.access` или любое из девяти разрешений вкладок.

Приложение Б. Данные в браузере (localStorage)

КЛЮЧ	СОДЕРЖИМОЕ
<code>task-control:session</code>	токен и пользователь текущего сеанса
<code>task-control:theme</code>	последняя тема компьютера
<code>task-control:theme:<employeeId></code>	тема сотрудника
<code>task-control:appearance:<employeeId></code>	личное оформление; прежний общий ключ <code>task-control:appearance</code> переносится к сотруднику
<code>task-control:notices</code> , <code>task-control:notices-since</code>	состояние уведомлений
<code>task-control:v...</code>	данные локального режима (тесты)

Приложение В. Подстановки шаблонов документов

«Планирование»: `{{отдел}}`, `{{неделя}}`, `{{год}}`, `{{с}}`, `{{по}}`, `{{дата}}`, `{{составил}}`, `{{всего}}`; блоки `{{#разделы}}` (`{{код}}`, `{{раздел}}`) → `{{#позиции}}` (`{{код}}`, `{{позиция}}`) → `{{#мероприятия}}` (`{{№}}`, `{{название}}`, `{{срок}}`, `{{исполнители}}`, `{{категория}}`, `{{статус}}`, `{{документ}}`, `{{результат}}`); `{{^мероприятия}}` / `{{^разделы}}` — текст для пустого списка.

«Отчётность»: то же плюс `{{получен}}`, `{{баллы}}` (общие и по разделу), `{{исполнено}}` у мероприятия, блок `{{#сотрудники}}` (`{{сотрудник}}`, `{{фιο}}`, `{{мероприятий}}`, `{{баллы}}`).

Приложение Г. Демонстрационные данные

- **Подразделения:** «Ведомство» → «Управление информационных технологий» → «Отдел разработки» с отделениями: Руководство, Проектная группа 1, Проектная группа 2, Группа документооборота, Отделение разработки, сопровождения, аналитики, тестирования.
- **Сотрудники:** 1–7 (администратор Иванов А.Б., руководитель Петров В.С., исполнители) и 40 сотрудников отдела разработки (101...); у всех Windows-логин вида `фамилия.ио`.
- **План:** четыре верхних раздела – «Организационные мероприятия», «Финансово-хозяйственная деятельность», «Проектная деятельность» (модули, внедрение, документация, сопровождение, безопасность, резервное копирование, инфраструктура), «Документооборот»; веса позиций 5 или 2.
- **Категории:** доклад руководству отдела / управления и службы / ведомства, промежуточный контроль; **виды отсутствий:** отпуск, командировка, отгул, больничный, учёба; **статусы:** «В работе», «Выполнено».
- **Шаблоны:** «План мероприятий на неделю», «Справка об исполнении».
- Задачи, отчёты, переписка, отсутствия и нормы строятся относительно текущей даты.

Приложение Д. Файловая карта ключевых модулей

ФАЙЛ	СОДЕРЖАНИЕ
<code>src/lib/types.ts</code>	модель данных
<code>src/lib/access.ts</code> , <code>permissions.ts</code>	разрешения, роли, права на задачи и отсутствия
<code>src/lib/reducer.ts</code>	все действия, права и миграции
<code>src/lib/logic.ts</code>	баллы, отчёты, оценка, поиск, проверки
<code>src/lib/dates.ts</code>	недели, колонки сроков, форматы
<code>src/lib/absences.ts</code>	остатки, пересечения, пик отпусков
<code>src/lib/notify.ts</code>	события уведомлений
<code>src/lib/templates.ts</code> , <code>planExport.ts</code> , <code>reportExport.ts</code> , <code>controlExport.ts</code> , <code>tetrisExport.ts</code> , <code>xlsx.ts</code> , <code>wordDoc.ts</code>	документы и выгрузки
<code>src/lib/appearance.ts</code> , <code>theme.ts</code> , <code>favicon.ts</code>	личное оформление и темы
<code>src/lib/StoreProvider.tsx</code> , <code>api.ts</code>	состояние клиента, обмен с сервером

ФАЙЛ	СОДЕРЖАНИЕ
<code>server/app.ts</code>	API, проверка данных, уведомления
<code>server/mssqlRepo.ts</code> , <code>pgRepo.ts</code> , <code>repo.ts</code> , <code>storage.ts</code>	хранилища
<code>server/windowsAuth.ts</code> , <code>activeDirectory.ts</code> , <code>auth.ts</code>	вход через Windows, каталог, токены
<code>server/iis.ts</code>	сервер IIS (статика + API)
<code>deploy-iis.bat</code> , <code>scripts/*</code>	публикация и сборка